

If you run `print(lines)`, you can see how we're storing information about lines in Python. Each line is stored as a list that contains the coordinates of two cities. For example, you can see the first line by running `print(lines[0])`, which will show you the following output:

```
[(0.21215859519373315, 0.1421890509660515), (0.25901824052776146,
0.4415438502354807)]
```

We can put these elements together in one function called `genlines` (short for “generate lines”), which takes `cities` and `itinerary` as arguments and returns a collection of lines connecting each city in our list of cities, in the order specified in the `itinerary`:

```
def genlines(cities,itinerary):
    lines = []
    for j in range(0,len(itinerary) - 1):
        lines.append([cities[itinerary[j]],cities[itinerary[j + 1]]])
    return(lines)
```

Now that we have a way to generate a collection of lines between each two cities in any itinerary, we can create a function that measures the total distances along those lines. It will start by defining our total distance as 0, and then for every element in our `lines` list, it will add the length of that line to the `distance` variable. We'll use the Pythagorean theorem to get these line lengths.

NOTE

Using the Pythagorean theorem to calculate distances on Earth is not quite correct; the surface of the Earth is curved, so more sophisticated geometry is required to get true distances between points on Earth. We're ignoring this small complexity and assuming that our salesman can burrow through the curved crust of the Earth to take direct routes, or else that he lives in some Flatland-esque geometrical utopia in which distances are easy to calculate using ancient Greek methods. Especially for short distances, the Pythagorean theorem provides a very good approximation to the true distance.

```
import math
def howfar(lines):
```